Decentralized Multi-Robot Collaboration: Scaling Vision-Language-Action Models from Single-Agent Manipulation to Communication-Free Teamwork

Workbook 2 Contribution Pack — Kanaka Sarat Siripurapu

CMPE 295A | Lane: Evaluation and data | Date: October 28, 2026

Advisor: Dr. Wencen Wu | Branch: feature/kanaka_workbook2

How to use this file (for the group)

Paste these sections into the shared Workbook 2 research document. Chapters 3–5 need almost no rewrite; I only listed the small sync notes so nobody reopens settled decisions. Chapters 6 and 7 are my filled evaluation / system-design writeups for the rubric. Chapters 8 and 9 carry the Workbook 1 text with every change from that submission marked in bold, including the Gantt and PERT for both semesters.

Owner map we already agreed: Bharat (Ch 1–2), Ritwik (Ch 3–4 / sim), Snehil (Ch 5 + model path), Kanaka / Sarat (evaluation + Ch 8–9 updates). Chapter 6 and the evaluation half of Chapter 7 are also on my lane.

Sync notes for Chapters 3–5 (no rewrite needed)

Chapter 3 — Identify Baseline Approaches. Keep Workbook 1 as written. The baseline is still the public lerobot/smolvla_base checkpoint with no fine-tuning. The only sync is that the single-frame contract test is no longer pending: on 2026-10-26 the checkpoint loaded at 450.0M and returned six numbers in the SO-101 joint order, chunk shape (1, 50, 6). That closes the “is this checkpoint valid for this arm?” gate from Chapter 3. B0 / B1 / B3 scoring still uses the same protocol.

Chapter 4 — Dependencies and Deliverables. Keep Workbook 1. One status update only: the Phase 1 simulator host for the next two weeks is a RunPod A40 so we are not blocked waiting on the shared on-campus workstation schedule. Training stays on the HPC3 H100 / A100 queues. The halt-risk list (RT-core GPU, Isaac 4.5 pin, single LeRobot version, HPC access, public checkpoint + dataset) is unchanged.

Chapter 5 — Project Architecture. Keep the two-machine split (HPC train / RT-core sim). The interface contract is confirmed: observation.images.front and observation.images.wrist at 480×640 map to SmolVLA’s camera1 / camera2; the third camera key is left unused. Language string from the dataset is “Grab orange and place into plate.” Checkpoint file remains the only artifact that crosses machines.

6. Evaluation Methodology

Everything here is scored by one evaluation script on one environment, so that a pretrained checkpoint, a fine-tuned checkpoint, and any external reference are directly comparable. Chapter 3 identified the baselines; this chapter is how they are measured, and what we have already executed.

Testbed setup

The evaluation runs on an RT-core GPU. For Phase 1 scoring that host is a RunPod A40; Dr. Wu’s on-campus workstation remains the later host once the shared schedule opens. The stack is Isaac Sim 4.5.0, Isaac Lab v2.1.1 (matched to 4.5), and LeIsaac for the SO-101 pick-orange scene. The policy is served by LeRobot’s async policy server and reached by LeIsaac’s scripts/evaluation/policy_inference.py over a

local host and port. Training, when it happens, is on the HPC3 H100 or A100 queue and is not part of the testbed; only the resulting checkpoint crosses over.

Every reported run records the environment it ran under: driver version, Isaac Sim version, Isaac Lab version, LeIsaac version, and the single LeRobot version used for both training and evaluation. Laptop-side Phase 1 checks used LeRobot 0.6.1. The pod installs whatever LeIsaac imports, that version is written into the lab log, and the same pin is used for B1 and B3. This is not bookkeeping. A published checkpoint on this exact task moved from 44% to 27% per-orange success on a LeRobot version change alone, so the version is part of the result.

Datasets used in evaluation

LightwheelAI/leisaac-pick-orange: 60 episodes, 36,293 frames, 30 fps, robot type so101_follower, six-float action and state (shoulder_pan.pos, shoulder_lift.pos, elbow_flex.pos, wrist_flex.pos, wrist_roll.pos, gripper.pos), two camera streams observation.images.front and observation.images.wrist at 480×640. Median episode length is about 18 s. Three oranges per episode. Task string in the dataset: "Grab orange and place into plate." Evaluation is closed-loop in the simulator, not scored against held-out dataset frames; the dataset supplies the task, the embodiment, and later the fine-tuning data. Sibling sets so101-pick-pen and so101-place-orange stay reference only.

Baseline approaches

From Chapter 3, run in this order:

• B0, random action. Sample the action space each step. The floor. Proves a later non-zero score means something.

• B1, SmolVLA zero-shot. lerobot/smolvla_base, no fine-tuning, served as --policy_type=lerobot-smolvla. The primary baseline. A score near zero is expected and acceptable, because the checkpoint has never seen this scene.

• B2, published same-task checkpoints (external, quoted not rerun). The wsagi/ACT-PickOrange checkpoint reports 44.0% per-orange (33/75, 95% CI [29.5%, 58.5%]) on Isaac Sim 5.1; shadowHokage/act_policy reports 18.3%. These are indicative context, not our numbers, and they ran on a different simulator build.

• B3, SmolVLA fine-tuned on the 60-episode set, scored by the same script.

Test parameters

Fixed now so all runs are comparable:

• Task LeIsaac-SO101-PickOrange-v0, or the -Direct-v0 variant where the programmatic success check is needed.

• --episode_length_s=120, --step_hz=30, cameras enabled, --device=cuda.

• Language instruction held constant: "Grab orange and place into plate" (dataset string; do not substitute a paraphrased prompt).

• Camera mapping: front → camera1, wrist → camera2. Third SmolVLA camera key left unused.

• --eval_rounds=5, at least three independent runs pooled. Single-run spread on this task is about ±40%, so one round is noise and is not reported.

• Seeds recorded per run and reused across checkpoints.

• --policy_action_horizon: first rollouts use 16; sweep later is 1 to 50 (smolvla_base returns at most 50 future actions per query). Chosen value is published.

Performance measures

• Per-orange success rate. Oranges placed on the plate divided by three times the number of episodes. The headline number, comparable to the published table.

• Per-episode count. Oranges placed per episode, 0 to 3, reported as a histogram. Required, not optional: the observed distribution on this task is bimodal, so a mean alone hides the behavior.

• 95% confidence interval on the per-orange rate, with numerator and denominator shown.

• Success is read from the environment's own check, not a criterion we invent. Open check with Ritwik: confirm -Direct-v0 matches the three-orange goal including return to rest before any number is published as final.

Experiments executed so far

1. Single-frame contract test — DONE (2026-10-26). One leisaac-pick-orange frame through lerobot/smolvla_base on the laptop (no simulator). Checkpoint loads at 450.0M and prints six numbers in the dataset joint order, chunk (1, 50, 6). Pass.

2. Dataset audit — DONE. Episode count, fps, cameras, action order, task string, and median length confirmed against the Hugging Face card and episode samples. Notes live under phase1/01_dataset_exploration/.

3. Evaluation wrapper skeleton — DONE (Kanaka). Sits on top of policy_inference.py, talks to the async policy server, writes one CSV row per episode (run ID, seed, round, checkpoint, horizon, oranges placed, termination reason, simulator version, LeRobot version, video path). B0 and B1 command plans are written so the first scored runs are not composed under time pressure.

4. B0 random-action and B1 zero-shot scored rollouts — READY, waiting on the A40 pod scene import and one confirmed rollout (Ritwik tracks 3–4). Protocol, seeds, and CSV schema will not change when those land.

5. Horizon sweep and B3 fine-tuned score — next, after B1 exists on the pinned LeRobot version.

Results table (Workbook 2)

Run	Policy	Per-orange success	Per-episode mean	95% CI	Horizon	Sim
Contract	smolvla_base, 1 frame	n/a (shape check)	n/a	n/a	chunk 50	laptop
B0	Random action	pending pod	—	—	n/a	Isaac 4.5.0
B1	SmolVLA zero-shot	pending pod	—	—	16 then sweep	Isaac 4.5.0
B2	ACT (external)	44.0% (33/75)	1.32	[29.5%, 58.5%]	70	Isaac 5.1
B3	SmolVLA fine-tuned	pending HPC	—	—	sweep	Isaac 4.5.0

A 0% in the B1 row is still a completed baseline. The milestone is that the row can be filled at all: the scene, the action format, the cameras, and the scorer agree end to end. Contract-test pass means that gate is no longer theoretical.

7. System Design / Methodology

Solution overview

The solution is a closed loop between a policy and a simulated robot, plus an offline training step that produces the policy. It reuses three existing systems (Isaac Lab, LeIsaac, LeRobot) and adds one component of our own: a thin evaluation wrapper that runs many seeded episodes and logs per-episode outcomes. The design goal is that a single checkpoint file is the only thing that changes between the baseline and the fine-tuned result, so any change in the score is attributable to training and nothing else.

Components

• Scene and robot (LeIsaac + Isaac Lab). Provides LeIsaac-SO101-PickOrange-v0: the SO-101 arm, the pick-orange table scene, the two cameras, and the programmatic success check (via the -Direct-v0 variant). We do not modify it.

• Policy server (LeRobot). lerobot.async_inference.policy_server holds a checkpoint and its normalization statistics and answers observation-to-action requests. Swapping the checkpoint path swaps the policy without touching anything else.

• Inference client (LeIsaac). scripts/evaluation/policy_inference.py steps the scene, sends observations to the policy server, and applies the returned actions. --policy_type=lerobot-smolvla is the path our model uses.

• Evaluation wrapper (ours — Kanaka). Runs the client for N rounds across M seeds, writes one row per episode, and prints the pooled per-orange rate with a confidence interval. This is the only scoring code we write.

• Trainer (LeRobot, on HPC). Fine-tunes SmolVLA on the 60-episode dataset and emits a checkpoint. Not part of the runtime loop.

Evaluation loop (pseudo-code)

```
load pinned versions (Isaac 4.5.0, Isaac Lab, LeIsaac, LeRobot)
start policy_server(checkpoint, norm_stats, device=cuda)
for run in 1..R:                    # R ≥ 3 pooled runs
  for round in 1..5:
    seed = recorded_seeds[run][round]
    reset LeIsaac-SO101-PickOrange-Direct-v0(seed)
    oranges = 0
    for t in 0 .. (120s * 30Hz):
      obs = {front→camera1, wrist→camera2, joint_state}
      if t % horizon == 0:
        action_chunk = policy_server.step(obs, language)
      apply action_chunk[t % horizon]   # 6 numbers, SO-101 order
      oranges = env.success_count()     # environment check only
    append CSV row (seed, oranges, versions, video)
print per-orange rate = sum(oranges) / (3 * episodes)
print histogram 0..3 and 95% CI
```

Algorithmic details that affect performance

• Action horizon. SmolVLA predicts a chunk of future actions; the client re-queries every --policy_action_horizon steps. First rollouts use 16; the sweep is 1–50 because smolvla_base returns at most 50. Leaving the default forever silently costs accuracy.

• Normalization statistics. The checkpoint's input/output normalization must travel with the weights. The wrapper logs which statistics were loaded.

• Version pinning. One LeRobot version for training and evaluation. A dataloader / loss change across versions moved a same-task checkpoint from 44% to 27%.

• Camera and language contract. Front/wrist map to camera1/camera2; unused camera3 is not fed a black image. Language is the dataset string exactly.

Deployment / reproduction steps

1. Clone the repo. Use branch feature/kanaka_workbook2 for this contribution; merge Ritwik's phase1/runbooks from feature/ritwik_workflow.

2. Laptop checks (no GPU): pip install -r phase1/requirements-phase1.txt, then run phase1/01_dataset_exploration/explore_dataset.py and phase1/02_model_exploration/contract_test.py. Expect six floats and chunk (1,50,6).

3. Simulator host: follow phase1/03_sim_setup/RUNBOOK.md on a RunPod A40. Record nvidia-smi before install. Image nvcr.io/nvidia/isaac-sim:4.5.0. Import check must find LeIsaac-SO101-PickOrange-v0. Write the LeRobot version into Lab_log.md.

4. First rollout: phase1/04_rollout/RUNBOOK.md. Confirm arm moves, action length 6, both cameras write frames to disk (headless).

5. Score: start the async policy server with lerobot/smolvla_base, run the evaluation wrapper with the recorded seeds, --policy_language_instruction="Grab orange and place into plate", horizon 16. Commit the CSV under phase1/05_evaluation/.

6. Fine-tune on HPC (~20k steps), copy checkpoint + norm stats to the sim host, rerun the same wrapper and seeds for B3.

8. Implementation Plan and Progress

Bold text marks every change from the Workbook 1 submission.

The implementation is broken into stages that match the two-machine architecture: environment first, then the offline model checks, then the simulator, then the closed loop, then training. The plan mirrors the team workflow. Progress is tracked in the table at the end and updated as each item lands.

Plan

Stage 0 — Environment and hardware. Confirm HPC3 training access. Secure a ray-tracing GPU for the simulator. Pull the Isaac Sim 4.5.0 container. Pin the driver/Isaac pairing. **Phase 1 scoring host is a RunPod A40 so sim work is not blocked on the shared on-campus schedule; Dr. Wu's workstation remains the later host.**

Stage 1 — Offline model checks (no simulator). Install LeRobot at the pinned version. Load lerobot/smolvla_base. Run one dataset frame through it and confirm the output is six numbers in the SO-101 joint order. **Done 2026-10-26: checkpoint loads at 450.0M, chunk (1, 50, 6), joint order**

confirmed. Laptop used LeRobot 0.6.1; pod pin still to be written into the lab log once LeIsaac imports.

Stage 2 — Simulator stand-up. Install Isaac Lab (matched to 4.5) and LeIsaac. Run a stock example unmodified. Load the SO-101 into the leisaac-pick-orange scene and confirm it is visible. **Runbooks written under phase1/03_sim_setup/. Next concrete step is the A40 pod import check.**

Stage 3 — One rollout. Drive the arm from a policy through LeIsaac's policy_inference.py. Confirm three things: the arm moves, the action is six numbers, both cameras render. **Headless frame-to-disk path documented in phase1/04_rollout/RUNBOOK.md; first live rollout pending pod.**

Stage 4 — Close the loop and score. Wire the evaluation wrapper around the inference script. Run B0 (random) and B1 (zero-shot SmolVLA) under the fixed protocol and print a pooled per-orange success rate. A 0% for B1 is a pass. **Wrapper skeleton, CSV schema, metric definitions, failure-label sheet, and B0/B1 command plans are ready on the evaluation lane (Kanaka). Scoring starts the same day the first rollout passes — we are not waiting on design work.**

Stage 5 — Training. Overfit 3–5 episodes on HPC as a smoke test (proves training runs). Then the full fine-tune (~20k steps). Score the fine-tuned checkpoint with the same wrapper and seeds. **Still gated on Stages 2–4, unchanged.**

Hard gate. If the simulator (Stage 2) is not running on schedule, the simulation side moves to MuJoCo through LeRobot's gym environments. The checkpoint, dataset, action format, and scoring protocol do not change. **Gate date moved to Nov 1 (was Oct 11) because Stage 1 finished early and the A40 path is already chosen; October was not spent fighting the g17 WebRTC path.**

Progress

Item	Stage	Status	Notes
HPC3 access, GPU job runs end to end	0	Done	A100 job ran and exited cleanly
Ray-tracing GPU secured for the simulator	0	Done	**Phase 1 host = RunPod A40; campus workstation later**
Isaac Sim 4.5.0 container pulled	0	Done	Pinned to 4.5 for driver 560.35.03
Driver / Isaac version pinning decided	0	Done	Isaac 5.x excluded on this driver
LeRobot installed at a pinned version	1	**Laptop done / pod pin open**	**0.6.1 on laptop; pod writes final pin**
SmolVLA loads; one frame → six numbers	1	**Done**	**Contract test passed 2026-10-26**
Dataset audit + task string locked	1	**Done**	**60 eps; instruction = dataset string**
Isaac Lab + LeIsaac installed on RT GPU	2	In progress	**A40 runbook ready; import check next**
Stock Isaac Lab example runs	2	Pending	Follows import check
SO-101 loaded and visible in pick-orange scene	2	Pending	Headless frames to disk
One rollout: arm moves, 6-dim action, cameras render	3	Pending	**Runbook written early**
Evaluation wrapper (seeded runs, per-episode logs)	4	**Done (skeleton)**	**CSV schema + B0/B1 plans ready**
Failure labeling sheet	4	**Done (empty)**	**Categories fixed; labels after first runs**
B0 random-action baseline scored	4	Pending	Starts day-of first rollout

B1 zero-shot SmolVLA baseline scored	4	Pending	0% is a pass
Overfit smoke test (3–5 episodes) on HPC	5	Pending	Not started until Stages 1–4 land
Full fine-tune (~20k steps) + scored	5	Pending	Same wrapper and seeds as B1

What is proven so far (updated). The training machine works, the Isaac Sim 4.5 image renders on an RT-core GPU, the dataset contract is audited, and the public SmolVLA checkpoint speaks SO-101 without a simulator. The evaluation wrapper and scoring protocol exist before the first closed-loop episode, which is why the evaluation lane is ahead of the Sprint 2 plan even while the pod import is still open. Remaining work is software integration on the A40: LeIsaac-on-4.5 import, one rollout, then B0/B1 the same day.

Reproduction. Each completed item is recorded in the lab log with the exact command and the environment versions, so the progress table above is backed by a runnable trail and not just checkmarks. **Phase 1 scripts and reports live under phase1/.**

9. Project Schedule

Bold text marks every change from the Workbook 1 submission. **This revision adds an explicit Gantt view and a PERT dependency view covering both CMPE 295A and CMPE 295B, with named owners on every row.**

The schedule is organized into five two-week sprints in Semester A (late September to the December 4 advisor demonstration) and four blocks in Semester B (January to May) for the second-arm / no-communication work. Each teammate owns one lane end to end. Class assignments are not tasks here; a few external dates are shown only as milestone anchors.

Team lanes (owner → responsibility)

• Ritwik — Simulator and integration. RunPod/A40 setup, Isaac Sim 4.5, LeIsaac scene, the closed loop, and HPC fine-tuning runs.

• Snehil — Model and inference. SmolVLA loading, the single-frame contract test, and the SmolVLA-over-LeRobot policy-server path.

• Kanaka Sarat — Evaluation and data. The evaluation wrapper, the metric definitions, the dataset notes, and the scored baselines.

• Bharat — Research and documentation. Reference-paper stack, report chapters, meeting logs, and reproduction README.

Semester A sprints (updated)

Sprint 1 (Sep 29 – Oct 11): Environment and contract — **complete.**

Owner	Task	Status
Ritwik	Stand up RT-core host; Isaac Sim 4.5 path proven on A40 class GPU	**Done / moved to RunPod A40 for Phase 1**
Snehil	Install LeRobot; load smolvla_base; one frame → six numbers	**Done 2026-10-26**
Kanaka, Bharat	Dataset notes; draft success definition and metric	**Done**
Bharat	Reference stack from team PPT; meeting log	Done

Milestone: **contract test passed; environment recorded in lab log.**

Sprint 2 (Oct 12 – Oct 25 → **extended to Nov 1**): Loop and baselines

Owner	Task	Status
Ritwik	Load SO-101 in pick-orange; one rollout (moves, 6-dim, cameras)	**In progress — runbook ready, pod next**
Snehil	Bring up SmolVLA policy-server path; first horizon=16	**Pending pod; camera map locked**
Kanaka	Build evaluation wrapper; score B0 and B1	**Wrapper done; B0/B1 day-of rollout**
Bharat	Draft report Ch 1–2; keep version pins documented	In progress

Milestone: B0 and B1 per-orange rates logged (0% for B1 is a pass). **Target Nov 1.**

Sprint 3 (**Nov 2 – Nov 15**): Fine-tune

Owner	Task
Ritwik	Overfit smoke test (3–5 episodes) on HPC; then full fine-tune (~20k steps)
Snehil	Sync fine-tuned checkpoint + norm stats to sim host; verify it loads
Kanaka	Score B3 on the same seeds; produce the baseline-vs-fine-tune table
Bharat	Write up methods/results so far; failure-video labeling begins

Milestone: B3 scored; pretrained vs fine-tuned rates side by side.

Sprint 4 (**Nov 16 – Nov 29**): RL fine-tune and iteration (conditional)

Owner	Task
Ritwik	B4 residual RL from the B3 warm start (only if Sprint 3 landed clean)
Snehil	Support RL reward/observation wiring; re-run horizon sweep
Kanaka	Score B4; lock the eval protocol (seeds, pooled runs, CI)
Bharat	Build the failure catalog; draft the results narrative

Milestone: B4 numbers in hand (or documented as slipped, per the gate).

Sprint 5 (**Nov 30 – Dec 4**): Freeze, present, demo

Owner	Task
Ritwik	Freeze and tag the baseline checkpoint; prepare the live demo
Snehil	Verify one-command reproduction of a scored run on a clean setup
Kanaka	Final baseline table with confidence intervals; commit all per-episode logs
Bharat	Finish reproduction README and report chapters; end-of-November results presentation

Milestone: results presented (end of Nov); prototype demonstrated to advisor (Dec 4).

Semester B blocks (new — CMPE 295B)

Semester B takes the single-arm checkpoint from December and asks whether two copies of those weights can collaborate with no messages, following the CHORUS shared-weight idea on our SO-101 stack.

Block	Dates	Owner focus	Deliverable
B1 Dual-arm scene	Jan 20 – Feb 14	Ritwik (scene), Snehil (two policy servers)	Two SO-101s in one Isaac scene; each sees only own cameras
B2 Role prompts	Feb 15 – Mar 14	Snehil + Kanaka	Identity prompts ("You are Robot A/B"); no radio channel in the loop
B3 Collab tasks	Mar 15 – Apr 18	Kanaka (metrics), Ritwik (tasks)	Handover + joint-lift success rates; reaction-time to teammate motion
B4 Report / Expo	Apr 19 – May 15	Bharat (writing), all (demo)	Final report, videos, Expo poster; optional workshop note

Gantt chart (both semesters)

Rows are work items. Cells mark active weeks. Owners in parentheses.

#	Work item (owner)	Oct	Nov	Dec	Jan	Feb	Mar	Apr	May
1	Isaac 4.5 / A40 stand-up (Ritwik)	■■	■						
2	Contract test + dataset audit (Snehil/Kanaka)	■■							
3	Eval wrapper + metric lock (Kanaka)	■■	■						
4	First rollout + cameras (Ritwik)	■	■■						
5	B0 / B1 scored baselines (Kanaka)		■■						
6	Policy-server path + horizon sweep (Snehil)		■■■						
7	HPC fine-tune ~20k steps (Ritwik)		■■■						
8	B3 score		■■	■					

	+ table (Kanaka)								
9	Optional B4 RL (Ritwik/Snehil)								
10	Docs, README, Dec 4 demo (Bharat/all)								
11	Dual-arm Isaac scene (Ritwik)								
12	Shared weights + role prompts (Snehil)								
13	Handover / lift eval protocol (Kanaka)								
14	No-comm collab results (all)								
15	Final report + Expo (Bharat/all)								

PERT / dependency view

Critical path (Semester A): HPC access → Isaac 4.5 pin → contract test → SO-101 rollout → B0/B1 score → HPC fine-tune → B3 score → Dec 4 demo.

Critical path (Semester B): frozen single-arm checkpoint → dual-arm scene → role-prompt inference → collab-task metrics → report / Expo.

ID	Predecessor(s)	Item	Owner
A1	—	HPC3 + RT-core host available	Ritwik
A2	A1	Isaac Sim 4.5.0 + LeIsaac import	Ritwik
A3	—	Dataset audit + metric draft	Kanaka
A4	—	SmolVLA contract test (6-dim)	Snehil
A5	A2, A4	One closed-loop rollout	Ritwik
A6	A3, A5	B0 + B1 scored (wrapper)	Kanaka
A7	A4, A6	HPC fine-tune (~20k)	Ritwik
A8	A7	Checkpoint + norm stats on sim host	Snehil
A9	A6, A8	B3 scored on same seeds	Kanaka
A10	A9	Docs freeze + Dec 4 advisor demo	Bharat / all
B1	A10	Second SO-101 in shared scene	Ritwik

B2	B1, A8	Two local policy copies + role prompts	Snehil
B3	B2	Handover / joint-lift eval	Kanaka
B4	B3	Final report, videos, Expo	Bharat / all

Key project milestones

Date	Milestone
Oct 26 (done)	**Contract test passed; dataset audited**
Nov 1	**Simulator go/no-go on A40 (else MuJoCo); B0/B1 target**
Nov 15	**Full fine-tune complete and scored (B3)**
Nov 29	RL fine-tune results (B4) or documented slip
End of Nov	Results presentation
Dec 4	Early prototype demonstrated to advisor
Feb 14	**Dual-arm scene running (295B)**
Apr 18	**No-communication collab numbers locked**
May 15	**Final report / Expo**

External anchors (not project tasks): Workbook 1 closed early October; **Workbook 2 due Oct 30**; report first chapters and the advisor demo fall on December 4. The sprints are placed so project work lands ahead of these, not on top of them.

Individual contribution note (Kanaka Sarat Siripurapu)

For this Workbook 2 pack I own the evaluation methodology (Chapter 6), the evaluation-wrapper half of the system design (Chapter 7), and the bolded updates to the implementation plan and the two-semester schedule (Chapters 8–9). On the code / notes side that means the metric lock, dataset audit fields the scorer needs, the wrapper skeleton and CSV schema, the B0/B1 run plans, the failure-label sheet, and keeping Chapters 3–5 in sync with the Oct 26 contract-test result. Ritwik owns the simulator host and rollout. Snehil owns the policy-server path. Bharat owns the literature and report glue. I am not claiming those lanes in this file.

Repo: https://github.com/bharatmanivannan/decentralized-multi-robot-collaboration/tree/feature/kanaka_workbook2

Generated 2026-10-28 for group merge into Workbook 2 (PDF submission).